TECHNICAL SPECIFICATION

AgentDNA Dashboard Portal

Architecture, API surface, page reference and user flows — source material for the public documentation site.

Repository: agentdna-dashboard

Type: React 19 + TypeScript single-page application (Vite)

Backends: Dashboard middleware API & Agent-admin API

Audience: documentation authors, integrators, new engineers

Status: derived from the current `soni/user-page` branch source

0. How to use this document

This document is a complete, self-contained description of the AgentDNA Dashboard Portal front-end. It is written to be turned directly into a Docusaurus documentation site — every section maps cleanly onto a docs page, and §16 proposes a concrete sidebar structure.

- **§1–§3** are conceptual — good candidates for *Introduction* / *Concepts* pages.
- **§4–§8** are architectural — *Architecture* section.
- **§9** is the page-by-page product reference — *User Guide* section.
- **§10–§12** are the API and auth contracts — *API Reference* section.
- **§13–§15** are operational — *Deployment & Configuration* section.

> **Scope note.** This describes the front-end only. Backend implementation details (databases, chain writes, threat-detection logic) are out of scope; the dashboard consumes them purely over HTTP. Where the front-end compensates for a missing backend capability, that is called out explicitly so the docs do not over-promise.

Contents

1. Product overview

The AgentDNA Dashboard Portal is a web-based monitoring and analytics interface for autonomous agent networks. An organization deploys agents; those agents talk to each other and to external tools (called *Apps* in the UI) in order to fulfil user-initiated tasks. Every one of those hops is recorded. The dashboard is the window onto that record.

What the product answers

QUESTION	WHERE IT IS ANSWERED
What is my agent network doing right now?	Home dashboard — KPI tiles, 30-day volume chart, live interaction ledger
Which agents and apps are most active, and which are risky?	Agents & Apps page — top-N cards, reliability score, threat counts
What happened during one specific task?	Intent detail + Flow graph — participants, ordered interactions, animated replay
Who talked to whom, and was it blocked?	Interactions ledger + interaction drawer
What is this agent allowed to do, and when did that change?	Agent detail — policy viewer and policy-history tab
Who is asking to deploy an agent or get access to one?	Requests page — creation and access approval queues
What has a particular person in my org been doing?	User detail — interactions, intents, threats, agents deployed

Two audiences, one app

- **Users** see their own activity, can browse the org's agents and apps, request an agent deployment, and request access to an existing agent.
- **Admins** additionally approve or reject those requests, create org users, manage per-user agent access, and upload/replace agent policies.

Role is carried on the session as `is_admin` and gates UI at render time (`ProtectedRoute` plus per-component checks). It is not a security boundary on its own — the backend enforces authorization.

2. Domain concepts & glossary

TERM	MEANING IN THIS PRODUCT
DID	Decentralized identifier — the primary key for every actor (agent, app, user). Long opaque string; the UI shortens it to `first12…last6` and resolves it to a human name wherever possible. Agent DIDs in this deployment start with `bafy`, which the client uses to distinguish agents from tools when the backend does not label the side.
Agent	An autonomous deployed actor owned by the org. Carries a name, a deployer, a creation time, a policy document, and running counters (interactions, threats, reliability score).
App / Tool	An external capability an agent calls (e.g. a weather service). Identified by `toolDID`; the UI derives a "provider" by taking the part of the tool name before the first dot. Labelled *Apps* in the interface, `Tool` in the type system.

TERM	MEANING IN THIS PRODUCT
Intent	One user-initiated task and the entire agent chain it triggers. Has an initiator, a start (and optionally end) time, a status (`completed` / `running` / `failed` / `pending`), a threat flag, a chain depth, and a provenance record ID.
Interaction	A single hop inside an intent: one actor sending to another at a point in time, with a threat verdict and an optional block type.
Policy	A Markdown or plain-text document attached to an agent (or user) describing what it may do. Versioned — each upload becomes a history entry with an `updateID` and timestamp.
Threat	An interaction the platform flagged as unsafe. Threat counts roll up to agents, apps, intents, users, and the org.
Score / reliability	A backend-computed 0–100 trust value per agent and app, rendered as a score bar.
Provenance record ID	Chain reference proving the intent's execution record.
Block / block chain data	The signed per-step record of an intent (`trigger` , `delegate` , `tool_call` , `execute` , `response` , `verify`), linked by `parent_block` . Powers the trace inspector.
Request	An approval workflow item. Two kinds: `deploy_agent` (create a new agent) and `agent_access` (grant a user access to an existing one).
Intent number	A client-side convenience: intents are sorted oldest-first and numbered #1, #2, … so humans can refer to them without quoting a hash. Purely presentational — every API call still uses the real `intentID` .

3. Technology stack

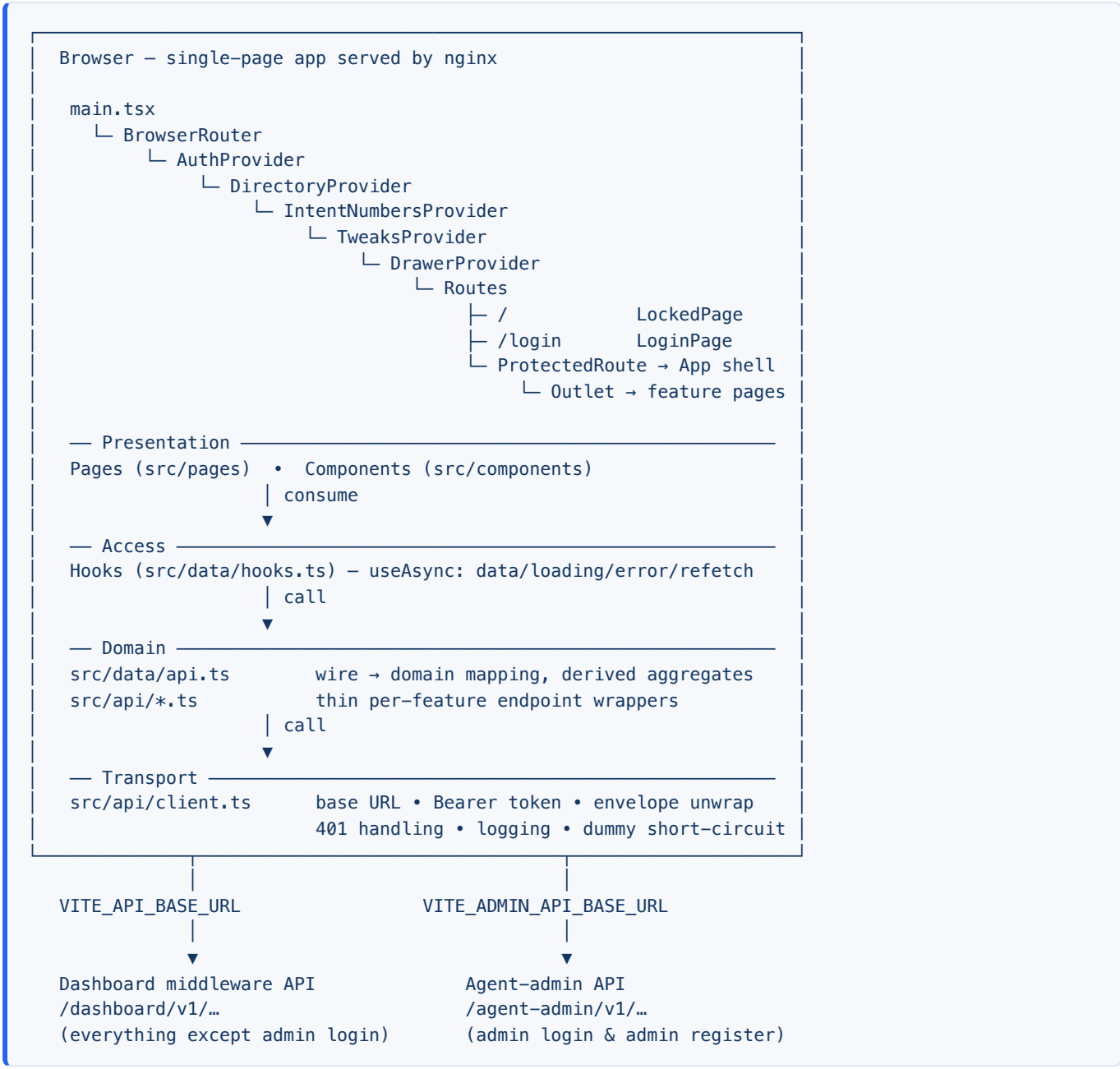
CONCERN	CHOICE	NOTES
UI framework	React 19	Function components + hooks only; `StrictMode` enabled
Language	TypeScript ~5.9	Strict project references (`tsconfig.app.json` / `tsconfig.node.json`)
Bundler / dev server	Vite 7	Dev port `4009` by default, `strictPort` , host exposed
Routing	react-router-dom 7	`BrowserRouter` + nested layout route
Styling	Tailwind CSS 4 + custom CSS	Tailwind via `@tailwindcss/vite` ; most visual identity lives in CSS custom properties in `src/index.css`
Charts	Recharts 3	Area/line/bar volume chart, sparklines
Animation	Framer Motion 12	Flow graph transitions, drawer/modal motion
Icons	lucide-react	Wrapped by `src/components/Icon.tsx` with a named icon registry
PDF generation	jsPDF 4	Client-side report export — no server involvement
Lint	ESLint 9 + typescript-eslint	Includes react-hooks and react-refresh plugins
Production server	nginx 1.27 (alpine)	Static files, SPA fallback, gzip, immutable asset caching
State management	React Context + local hooks	Deliberately no Redux / Zustand / React Query

NPM scripts

SCRIPT	COMMAND	PURPOSE
`npm run dev`	`vite`	Dev server with HMR
`npm run build`	`tsc -b && vite build`	Type-check then bundle to `dist/`
`npm run preview`	`vite preview`	Serve the production bundle locally
`npm run lint`	`eslint .`	Lint the whole repo

4. System architecture

The dashboard is a stateless client. It owns no persistence beyond browser `localStorage` (session token, cached user, UI preferences). Everything displayed comes from two HTTP services.



Layering contract

1. **Pages never call `fetch`.** They call a hook or a data-layer function. No page constructs a URL or knows about the token.
2. **`src/data/api.ts` owns every wire→domain mapping.** If the backend renames a field, that file is the only place that should need editing.
3. **`src/api/client.ts` is the only transport.** It is the single place that knows the base URL, the auth header, and the response envelope.
4. **Types flow one way.** Wire interfaces (`ApiAgent`, `ApiIntent`, …) are private to the data layer; the rest of the app only ever sees the domain types from `src/types.ts`.

Request lifecycle

Every call goes through `apiRequest<T>(path, opts)`:

1. **Dummy short-circuit** — if `VITE_DUMMY` is on and the mock router recognizes the path, the mock response is returned immediately and logged as `[DUMMY …]`.

2. **URL assembly** — `new URL(BASE + path)`; query params are appended, skipping `null` / `undefined` /empty values.
3. **Headers** — always `Accept: application/json`; `Content-Type: application/json` only when a body is present; `Authorization: Bearer <token>` unless `auth: false`.
4. **Timing & logging** — start time is captured and every outcome (success, network failure, non-OK, invalid JSON) is logged with elapsed milliseconds. This is intentional: the console is the primary field-debugging tool.
5. **401 handling** — throws `ApiError("Unauthorized", 401)`. If `skipLogoutOn401` is false, the token is cleared and the registered unauthorized handler fires, signing the user out.
6. **Envelope unwrap** — the body is parsed as `{ status, data, message }`. A non-OK HTTP status *or* `status: false` throws `ApiError(message, httpStatus)`. Otherwise the inner `data` is returned.

File uploads use `apiUpload<T>(path, formData)`, which is the same pipeline minus the `Content-Type` header, so the browser can set the multipart boundary itself.

> **Error model.** A single `ApiError` class carries `message` and `status`. Network failures surface as status `0`. Many read paths in the data layer deliberately swallow errors and return an empty result so one failing panel cannot blank the whole page; write paths always propagate so the UI can show a real message.

5. Application bootstrap & provider tree

`src/main.tsx` mounts the app and declares the entire route table. The provider order matters:

PROVIDER	RESPONSIBILITY	WHY AT THIS LEVEL
`AuthProvider`	Session, token storage, JWT decode/expiry, login/register/logout	Outermost — everything else may depend on who is signed in
`DirectoryProvider`	Builds a `DID → { name, kind }` map by walking every page of agents, tools and users	Needs auth; every table below it wants name resolution
`IntentNumbersProvider`	Walks every intent and assigns stable sequential numbers (oldest = #1)	Needs auth; used by intent chips everywhere
`TweaksProvider`	UI preferences — density, sidebar state, chart style, font — persisted to `localStorage`	Pure presentation, no data dependency
`DrawerProvider`	Global right-hand detail drawer: `{ kind, entity }` plus open/close	Innermost — the drawer is rendered by the app shell

The authenticated area is a single nested route: `<ProtectedRoute><App /></ProtectedRoute>`. `App.tsx` renders the persistent chrome and an `<Outlet />` for the active page.

App shell (`App.tsx`)

- **Sidebar** — logo, a *Workspace* group (Home, Intents, Agents & Apps, Flow, Requests, Interactions) and an *Account* group (Profile), plus a footer showing the signed-in identity and org. Collapsible; the collapsed state is a tweak.
- **Requests badge** — for admins only, the shell fetches page 1 of both request queues on mount and shows the combined count of `pending` items next to the Requests nav item.
- **Topbar** — sidebar toggle, breadcrumb (`ORG / Section` , derived from the current path), and global search.
- **Global search** — 300 ms debounce, hits `/search?q=` , renders a `SearchDropdown` grouped into Agents / Apps / Intents; selecting a result navigates and clears the query. Closes on outside click or `Escape` .
- **Drawer host** — one `<Drawer>` whose body is chosen from the drawer kind: `agent` / `tool` → `EntityDetail` , `interaction` → `InteractionDetail` , `intent` → `IntentDetail` .
- **Font injection** — the selected tweak font is written to the `--font-body` CSS variable on the document element.

6. Repository structure

```
agentdna-dashboard/
├── docker/
│   ├── Dockerfile            multi-stage node build → nginx serve
│   └── entrypoint.sh         writes env-config.js at container start
├── nginx.conf                reference SPA config (SPA fallback, gzip, caching)
├── vite.config.ts            react + tailwind plugins, dev port, allowed hosts
├── index.html                loads /env-config.js before the bundle
├── public/
└── src/
    ├── main.tsx              provider tree + route table
    ├── App.tsx               authenticated shell
    ├── types.ts              domain types
    ├── index.css             design tokens + component styles
    │
    ├── api/                  transport + thin per-feature wrappers
    │   ├── client.ts         apiRequest / apiUpload / token / ApiError
    │   ├── auth.ts           login, admin login, register, OTP, password reset
    │   ├── profile.ts        user & admin profile, update, change password
    │   ├── users.ts          org users, create user, access grant/revoke
    │   ├── requests.ts       agent-creation & agent-access workflows
    │   ├── policy.ts         policy fetch/upload/history
    │   └── keys.ts           API key lifecycle, token usage
    │
    ├── data/
    │   ├── api.ts            domain data layer + wire→domain mappers
    │   ├── hooks.ts          useAsync + all useX hooks
    │   ├── dummyRouter.ts    offline mock router
    │   └── dummy.json        seed data
    │
    ├── context/
    │   ├── AuthContext.tsx
    │   ├── DirectoryContext.tsx
    │   ├── IntentNumbersContext.tsx
    │   ├── DrawerContext.tsx
    │   └── TweaksContext.tsx
    │
    ├── components/           shared UI
    │   ├── drawer/           EntityDetail, InteractionDetail, IntentDetail, DrawerSection
    │   └── forms/            modals & pickers
    │
    ├── lib/
    │   ├── exportAgentPdf.ts • exportIntentPdf.ts • exportListPdf.ts
    │   └── format.ts         timeAgo, fmtRuntime, initials
    │
    └── pages/
        ├── LockedPage.tsx    public landing + auth
        ├── LoginPage.tsx
        ├── HomePage.tsx
        ├── IntentsPage.tsx • IntentDetailPage.tsx
        ├── AgentsToolsPage.tsx • AgentDetailPage.tsx • ToolDetailPage.tsx
        ├── UserDetailPage.tsx
        ├── InteractionsPage.tsx • AlertsPage.tsx
        ├── RequestsPage.tsx • requests/UsersTab.tsx
        ├── ProfilePage.tsx
        └── flow/             FlowPage.tsx • FlowCanvas.tsx • flowData.ts
```

7. Routing map

PATH	COMPONENT	ACCESS	PURPOSE
`/`	LockedPage	PUBLIC	Landing page with live global stats and the full auth experience
`/login`	LoginPage	PUBLIC	Standalone sign-in screen
`/dashboard`	HomePage	AUTH	Org overview
`/intents`	IntentsPage	AUTH	Intent list
`/intents/:intentId`	IntentDetailPage	AUTH	One intent in depth
`/agents`	AgentsToolsPage	AUTH	Agents and Apps inventory
`/agents/:agentId`	AgentDetailPage	AUTH	One agent in depth
`/tools/:toolId`	ToolDetailPage	AUTH	One app in depth (param may be a name or a DID)
`/users/:userId`	UserDetailPage	AUTH	One org user in depth
`/graph`	FlowPage	AUTH	Flow graph, no intent selected
`/graph/:intentId`	FlowPage	AUTH	Animated replay of one intent
`/requests`	RequestsPage	AUTH	Approval queues; the Users tab is admin-only
`/interactions`	InteractionsPage	AUTH	Full interaction ledger
`/profile`	ProfilePage	AUTH	Account, API key, usage, security
`*`	→ `/dashboard`	AUTH	Catch-all redirect

Unauthenticated visitors hitting a protected path are redirected to `/` with the attempted path preserved in router state, so they land where they meant to go after signing in. `AlertsPage` and `LandingPage` exist in the tree but are not currently routed.

8. State management & contexts

8.1 AuthContext

Owns the session. Exposes `user`, `token`, `loading`, and the actions `login`, `loginAdmin`, `registerAdmin`, `registerUser`, `logout`, `patchUser`.

- Persists the JWT at `localStorage["agentdna.token"]` and the decoded user at `localStorage["agentdna.user"]`.
- On boot it restores the stored user, falling back to decoding the stored token.
- `decodeJwt` base64url-decodes the payload. A token whose `exp` has passed yields `null` — an expired token is never treated as a session.
- A 30-second interval re-checks the token and signs the user out the moment it expires, rather than waiting for the next 401.
- Registers itself as the client's unauthorized handler so a hard 401 clears the session.
- Admin JWTs carry `sub` (username) and no `email`; user JWTs carry `email`. When `is_admin` is absent the context infers admin from that shape.

- After any successful sign-in it fetches the matching profile in the background and patches `name` and `org_id` onto the session.

8.2 DirectoryContext

Interaction records carry DIDs, not names. On mount this provider walks *every page* of `/agents-list`, `/tools-list` and `/users-list` (capped at 200 pages each, stopping early on an empty page or when `totalPages` is reached) and builds a `Map<DID, { name, kind }>`. Each of the three fetches fails independently and degrades to an empty list rather than breaking the map.

`useResolveName()` returns a lookup that falls back to a shortened DID for unknown entries. Tables prefer the directory name, then any backend-supplied name on the record, then the shortened DID.

8.3 IntentNumbersContext

Loads all intents once, sorts oldest-first, and exposes `Map<intentID, number>`. Also exports `IntentIdChip`, which renders a truncated ID (`abcd...xyz`) with a hover tooltip showing the full value, positioned in the viewport so it is never clipped by a table's overflow.

8.4 DrawerContext & TweaksContext

`DrawerContext` holds at most one `{ kind, entity }` pair; any table row can open a detail panel without prop-drilling. `TweaksContext` holds four presentation preferences — density (`compact` / `comfortable` / `spacious`), sidebar (`expanded` / `collapsed`), chart style (`area` / `line` / `bar`) and font (Inter, Geist, IBM Plex Sans, Manrope, DM Sans) — persisted to `localStorage["agentdna.tweaks"]`.

8.5 The `useAsync` primitive

```
interface AsyncState<T> {
  data: T;              // always defined — callers supply an initial value
  loading: boolean;
  error: Error | null;
  refetch: () => void;  // bumps an internal nonce to re-run the effect
}
```

Every data hook is one line on top of this. Stale responses are discarded via a `cancelled` flag, so rapid pagination cannot render an out-of-order page.

> **No shared cache.** There is no request de-duplication or cache layer. Navigating away and back refetches. This keeps the mental model simple and the data fresh, at the cost of repeat traffic — worth stating plainly in the docs so integrators size their backend accordingly.

9. Page reference

9.1 Landing & authentication — `/` (LockedPage)

The public entry point. Combines marketing surface with the full auth experience.

- **Live stats strip** — `/global-stats` (unauthenticated) supplies Agents, Users, Interactions, Intents and Threats, abbreviated as `1.2K` / `3.4M`.
- **Role switch** — User or Admin.
- **Sign in** — user sign-in goes to the dashboard API; admin sign-in goes to the agent-admin API.
- **Register** (user role only) — email, name, password, confirm, and an emailed OTP. `Send OTP` starts a 300-second countdown before a resend is allowed.
- **Forgot password** — a separate screen: request an OTP, then submit OTP + new password.
- **Post-auth redirect** — an already-signed-in visitor is redirected to the path they originally attempted, or `/dashboard`.

Default org for self-service user registration is `AGENT_DNA_BETA`.

9.2 Home — `/dashboard`

Data sources	`/home-metrics`, `/interactions-list`, `/interactions/series?range=30d`, `/agents-apps-metrics`
Empty state	When `agentCount` is 0 after loading, the page replaces itself with a "get started" panel instead of showing zeroed tiles

- **KPI tiles** — Active Agents, Total Intents, Total Interactions, Threats Detected.
- **Volume chart** — 30-day series split into safe vs threat, with total derived client-side. Renders as area, line or bar according to the chart-style tweak.
- **Top agents / Top apps** — a toggle between the two leaderboards from `/agents-apps-metrics`; rows link through to the relevant detail page.
- **Bottom ledger** — tabbed *Interactions* / *Threats*. Interactions are server-paginated; threats are the same feed filtered client-side on the threat flag. Clicking a row opens the interaction drawer.
- **CSV export** — builds a two-section CSV (summary block, then the agent list) entirely in the browser and downloads it as `agentdna-dashboard-YYYY-MM-DD.csv`.

9.3 Intents — `/intents`

Server-paginated intent table with client-side filter pills: *All*, *Threats*, *Safe*.

Columns: Intent ID (truncated chip with hover tooltip), Status chip (`completed` green, `running` blue, `failed` red, `pending` amber), Initiator, Agents/Apps touched, Interactions, Threats, Started. Header cells with a comparator are sortable. Summary tiles above the table aggregate the current page.

> The filter pills operate on the *current page only*, because filtering is not pushed to the API. Document this so users are not surprised when "Threats" shows fewer rows than the org total.

9.4 Intent detail — `/intents/:intentId`

Data sources	`/intent-info` + paginated `/interactions-list?intentID=`
Tabs	**Interactions** (paginated ledger) · **Agents & Apps** (participants)
Tiles	Interactions · Agents touched · Apps touched · Threats
Actions	Export a PDF report · jump to the Flow view · open any row in the drawer

Participant rows are derived, not fetched: the client walks every page of the intent's interactions, drops the initiator, groups the remaining actors by DID, and counts appearances, threats and last-seen time. Agent-vs-tool classification uses the `bafy` DID prefix. Clicking a participant goes to the agent detail page or opens the tool drawer.

Runtime is computed as `endedAt − startedAt`; an intent with no `endedAt` shows a runtime of zero.

9.5 Agents & Apps — `/agents`

- **Header cards** — Top Agents and Top Apps by interaction volume, plus org totals and average reliability from `/agents-apps-metrics`.
- **Tabbed table** — *Agents* and *Apps*, each server-paginated with its own page state. Agent rows navigate to `/agents/:id`; app rows navigate to `/tools/:name` (URL-encoded).
- **Actions** — request an agent deployment (`AgentRequestModal`), request access to an existing agent (`AccessRequestModal`), and export the visible list as a PDF.

9.6 Agent detail — `/agents/:agentId`

Data sources	`/agent-info`, `/agent-interactions`, `/agent-intents`, `/agent-policy-history`, `/agent-policy-update`
Tabs	**Interactions** · **Intents** · **Policy History**
Tiles	Interactions · Threats · Intents handled (plus reliability score bar)

- **Header** — name, shortened DID with copy, deployer, org, created-ago, status chip.
- **Policy viewer** — the current policy text from `/agent-info`; empty when none is uploaded. Admins can replace it via `EditAgentPolicyModal` (multipart `.md` / `.txt` upload).
- **Policy History** — history entries sorted oldest-first to assign a change number, then displayed newest-first. Selecting an entry fetches that version's full text and opens it in `ViewPolicyModal`.
- **PDF export** — one report bundling the agent summary, interactions, intents and policy history.

9.7 App detail — `/tools/:toolId`

The route parameter can be either a DID or a tool name; the data layer inspects it (`bafy` prefix or a `did:` substring means DID) and sends `toolDID` or `name` accordingly. A single `/tool-info` call returns the summary plus both paginated sub-collections, so the *Interactions* and *Intents* tabs each drive their own page parameter on the same endpoint.

9.8 User detail — `/users/:userId`

One `/user-info` call returns the user record plus four independently paginated collections. Tabs: **Interactions**, **Intents**, **Threats**, **Agents Deployed** — each with its own page parameter, all sent on every request. Header

stats: interactions, threats, intents, agents deployed, agents accessible, active flag, created and last-active times.

9.9 Interactions — `/interactions`

The complete org ledger, server-paginated. The `LedgerTable` columns are: Interaction ID, Initiator, Interacted with, Intent, Threat, Time. Every row opens the interaction drawer, which shows the full IDs, both counterparties, the intent link, the threat verdict, the block type and the timestamp.

9.10 Flow — `/graph/:intentId`

The most involved screen: an animated reconstruction of how an intent moved through the network.

Source precedence

1. **`/intent-diagram`** — preferred. Carries real messages, typed hops (`trigger`, `delegate`, `tool_call`, `response`, `execute`) and correct tree structure.
2. **`/interactions-list` + `/intent-block-data`** — fallback. The flow graph is built from the interaction list, and the trace is enriched by flattening the `parent_block` chain into an oldest→newest sequence.

When the diagram endpoint answers, its trace is *not* overridden by block data — block messages are placeholders and would degrade the display.

Model

```
Flow {
  intentId, intent,
  nodes: FlowNode[]        // kind: human | agent | tool, tier-laid-out x/y
  edges: [from, to][]      // unique directed pairs
  steps: FlowStep[]        // ordered hops: dir (request|response), title, summary,
                           // verdict (allowed|blocked), checks {identity,trust,scope},
                           // latency, spanId
  status: "halted" | "completed"
  trace: FlowTrace         // span tree + totals (tokens in/out, cost, ids)
  rawDiagram?: unknown     // raw API payload, shown verbatim in the JSON tab
}
```

Interaction model

- **Canvas** — nodes laid out in tiers (initiator → agents → tools), edges highlighted as the active step advances, threat hops marked.
- **Step rail** — a scrollable list of hops; the active card auto-scrolls into view. Clicking a step jumps to it and pauses playback.
- **Playback** — auto-advance every 2.4 s, wrapping at the end. The current step index is persisted to `localStorage["flow.step"]` and clamped when the flow changes.
- **TraceInspector** — an OpenTelemetry-style span tree per step: input, output, model, status, signature, metadata, plus a raw-JSON tab.

9.11 Requests — `/requests`

TAB	VISIBLE TO	ENDPOINT
Creation requests	All	`/agents-creation-requests-list` (admin) or `…-list-user`
Org access requests	All	`/agent-access-requests-list-org`
My access requests	All	`/agent-access-requests-list-user`

TAB	VISIBLE TO	ENDPOINT
Users	ADMIN	`/users-list`

- Rows show request type, agent name, requester (DID resolved to a name), status chip (pending / approved / rejected), created time, and the attached policy.
- Admins get approve and reject actions per row; the list reloads afterwards.
- Users can create a new agent request and edit their own pending ones.
- Approving a deployment opens `DeployAgentModal`, which shows the deployment phase (loading / success / error) rather than a bare toast.
- A backend `no_did` error is handled as its own empty state — the account has no DID yet, which is a distinct situation from "no requests".
- The Users tab lists org members with intent and threat counts and accessible-agent counts, offers `AddUserModal` for admins, and opens `UserAccessDrawer` to manage per-agent access.

9.12 Profile — `/profile`

Loads `/admin-profile` or `/user-profile` depending on the role, plus `/token-usage`. Shows name, email, organization, role, member-since, and the API key — masked as `abcd••••••••wxyz` with reveal and copy actions. Token usage renders as a used/limit meter with a reset date. Inline editing of name and email posts to `/update-profile`; a separate form posts to `/change-password`. A support contact block and sign-out complete the page.

10. Data layer & type mapping

10.1 Domain types

```
type Status = "safe" | "warn" | "threat";

interface Agent {
  id; name; score; created;        // created = minutes since creation
  interactions; threats; connected;
  status: Status; env; owner;      // env = orgID, owner = deployer DID
  policy?: string;                 // raw .md/.txt text, "" when none
}

interface Tool {
  id; name; score; created;
  interactions; threats; connected;
  status: Status; scope; provider; // provider = tool name before the first "."
}

interface Intent {
  id; name;                        // name carries the status string
  initiator: Agent;                // stub Agent built from the initiator DID
  runtime;                         // ms, endedAt − startedAt (0 if unfinished)
  started;                         // minutes ago
  agentsInteracted; toolsInteracted; interactionsCount;
  threats; score; status: Status;
  provenanceRecordID; signature?;
}

interface Interaction {
  id; initiator: EntityRef; target: EntityRef;
  targetType: "agent" | "tool";
  intent: { id; name };
  runtime; threat: boolean; created;  // created = minutes ago
  blockType?: string;
}
```

Also defined: `TimeSeries`, `HeatmapRow`, `LogEntry`, `IntentParticipant`, `HomeMetrics`, `HomeAgentSummary`, `PublicMetrics`.

10.2 Mapping conventions

CONVENTION	DETAIL
Timestamps → minutes-ago	`isoToMinutesAgo` converts every ISO timestamp to an integer number of minutes at map time. The UI formats it with `timeAgo`. Invalid or missing dates become `0`.
DID shortening	Anything longer than 24 characters renders as `first12…last6`.
Name resolution order	Directory map → backend-supplied `fromName` / `toName` → shortened DID.
Derived status	Agents flip to `warn` above 5 threats; tools above 4. These thresholds live in the client.
Agent vs tool	Determined by the `bafy` DID prefix wherever the backend does not label the side.
Full-collection walks	`fetchAllAgents`, `fetchAllTools`, `fetchAllIntents`, `listAllUsers` loop pages up to a 200-page ceiling, stopping on an empty page or at `totalPages`.

CONVENTION	DETAIL
Defensive reads	Detail and series fetches catch and return `null` or an empty result so a single failing panel does not blank the page.

10.3 Hook catalogue

HOOK	RETURNS
`useHomeMetrics(page)`	Home KPI payload + agent summary list
`useAgentsAppsMetrics()`	Top agents/apps and org rollups
`useSeries(range)`	Interaction time series for `24h` / `7d` / `30d`
`useAgents / useAgentsPaged(page)`	Agent list, plain or with pagination metadata
`useTools / useToolsPaged(page)`	App list
`useIntents / useIntentsPaged(page)`	Intent list
`useInteractions / useInteractionsPaged(page)`	Interaction ledger
`useAlerts(page)`	Interactions filtered to threats (client-side)
`useUsers(page)`	Org users
`useAgent(id)`	Single agent
`useAgentInteractions / useAgentIntents(id)`	Agent sub-collections
`useAgentPolicyHistory(id)`	Policy version list
`useIntent(id)`	Intent header with derived participant counts
`useIntentInteractionsPaged(id, page)`	Interactions inside one intent
`useIntentParticipants(id)`	Derived participant rollup
`useIntentDiagram(id) / useIntentBlockData(id)`	Flow graph sources
`useToolInfo(nameOrDid, …)`	App detail with both sub-collections
`useUserInfo(userID, …4 page params)`	User detail with all four sub-collections

`useHeatmap` and `useLogs` exist but currently resolve to empty arrays — no backend endpoint yet.

11. API reference

11.1 Conventions

- **Base URLs** — everything except admin sign-in/registration is relative to `VITE_API_BASE_URL`; admin sign-in uses `VITE_ADMIN_API_BASE_URL`.
- **Envelope** — every response is `{ status: boolean, data: T, message?: string }`. The client returns `data` and throws on `status: false`.
- **Auth** — `Authorization: Bearer <jwt>` on all authenticated calls.
- **Pagination** — list responses carry `{ <items>, total, page, pageSize, totalPages }`. Default page size is 10.
- **Uploads** — multipart `FormData`; the client omits `Content-Type` so the browser sets the boundary.

11.2 Authentication & account

METHOD	ENDPOINT	AUTH	REQUEST → RESPONSE
POST	`/login`	NO	`{ email, password }` → `{ token, did, email, org_id, api_key, nft_id?, is_admin, agent_access_list? }`
POST	`/login` *(admin base)*	NO	`{ username, password }` → JWT string in `data`
POST	`/register-user`	NO	`{ name?, email, password, orgID, otp }` → `{ api_key, name, email, orgID }`
POST	`/create-admin`	NO	`{ username, email, orgID, password, otp }` → `{ did }`
POST	`/register-admin`	NO	`{ did, org_id }` → same. Whitelists a newly created admin DID in the middleware.
POST	`/send-otp`	NO	`{ email }` → `{ message }`
POST	`/forgot-password`	NO	`{ email }` → `{ message }`
POST	`/reset-password`	NO	`{ email, otp, new_password }` → `{ message }`
GET	`/user-profile`	YES	→ `{ name, email, apiKey, organizationID, createdAt, adminEmail }`
GET	`/admin-profile`	ADMIN	→ `{ name, email, organizationID, apiKey, agentCount, intentCount, threatCount, totalUsers, createdAt }` (`createdAt` = epoch seconds)
PATCH	`/update-profile`	YES	`{ name?, email? }` → `{ message }`
POST	`/change-password`	YES	`{ currentPassword, newPassword }` → `{ message }`
POST	`/generate-api-key`	YES	→ `{ api_key }`
POST	`/revoke-api-key`	YES	→ void
GET	`/token-usage`	YES	→ `{ tokensUsed, tokensLimit, resetAt? }`

11.3 Metrics

METHOD	ENDPOINT	RESPONSE & CONSUMER
GET	`/global-stats`	`{ totalUsers, totalAgents, totalInteractions, totalIntents, totalThreats }` — public landing page, no auth
GET	`/home-metrics?page=`	`{ agentCount, intentCount, interactionsCount, threatCount, page, agentList[] }` where each entry is `{ agentID, agentName, totalInteractions, totalThreats }`
GET	`/agents-apps-metrics`	`{ topAgents[], topApps[], metrics: { totalInteractions, totalThreats, totalAgents, totalApps, avgReliability } }`
GET	`/interactions/series?range=`	`{ safe: number[], threats: number[] }`; `total` is summed client-side. `range` ∈ `24h` | `7d` | `30d`. Failure degrades to empty arrays.

11.4 Lists

METHOD	ENDPOINT	ITEM SHAPE
GET	`/agents-list?page=`	`agentsList[]`: `{ agentID, agentName, createdAt, deployer, policy, totalInteractions, totalThreats, score }`
GET	`/tools-list?page=`	`toolsList[]`: `{ toolDID, toolName, totalInteractions, totalThreats, score }`
GET	`/intent-list?page=`	`intentsList[]`: `{ intentID, initiatorDID, initiatorName?, startedAt, endedAt?, status, threatDetected, threatCount?, flowType?, executor?, chainDepth?, interactionsCount?, agentsCount?, toolsCount?, provenanceRecordID? }`
GET	`/interactions-list?page=&intentID=`	`interactionList[]`: `{ interactionID, from, to, fromName?, toName?, threat, intentID, time, blockType? }`. `intentID` is optional and scopes the feed to one intent.
GET	`/users-list?page=`	`usersList[]`: `{ userID, userName, createdAt, totalIntents, totalThreats, accessAgentCount }`
GET	`/search?q=`	`{ agents: [{ did, name, orgID }], apps: [{ did, name }], intents: [{ intentID, flowType, status, threatDetected, startedAt }] }`

11.5 Detail

METHOD	ENDPOINT	RESPONSE
GET	`/agent-info?agentDID=`	`{ agentDID, agentName, createdAt, deployerDID, policy?, orgID, totalInteractions, totalThreats, score }`
GET	`/agent-interactions?agentDID=&page=`	`interactionsList[]` + pagination
GET	`/agent-intents?agentDID=&page=`	`intentsList[]` + pagination

METHOD	ENDPOINT	RESPONSE
GET	`/tool-info?toolDID=|name=&interactionsPage=&intentsPage=`	`{ toolDID, toolName, totalInteractions, totalThreats, totalIntents, totalAgents, score, interactions:{list,…}, intents:{list,…} }`
GET	`/user-info?userID=&interactionsPage=&intentsPage=&threatsPage=&agentsPage=`	`{ user:{…}, interactions:{…}, intents:{…}, threats:{…}, agents:{…} }` — four independently paginated collections in one call
GET	`/intent-info?intentID=`	`{ intentID, initiatorDID, initiatorName?, startedAt, endedAt?, status, threatDetected, provenanceRecordID?, interactions? }`
GET	`/intent-diagram?intentID=`	`{ basicInfo: { intentID, initiatorDID, initiatorName, flowType, status, threatDetected, chainDepth, interactionsCount, agentsCount, toolsCount, startedAt }, interactions: [{ interactionID, initiator, initiatorName, to, toName, type, message, intentID, threat, epoch }] }`
GET	`/intent-block-data?intent_id=`	A recursive `IntentBlock` : `{ id, block_index, agent_did, agent_name, direction, block_type, message, response, delegate_to, received_from, cbac_app, cbac_decision, threat_detected, trust_issues[], signature, created_at, parent_block }`

Note the parameter casing inconsistency: `/intent-block-data` takes `intent_id` (snake case) while every other intent endpoint takes `intentID`.

11.6 Policy

METHOD	ENDPOINT	PURPOSE
GET	`/agent-policy-history?agentDID=`	`{ agentDID?, nftID?, history: [{ updateID, time }] }` — `time` is epoch seconds
GET	`/agent-policy-update?agentDID=&updateID=`	`{ updateID, time, policy }` — full text of one version
GET	`/user-policy?userDID=`	`{ filename?, content, uploadedAt? }`
POST	`/upload-agent-policy`	multipart `agentDID` + `file` (`.md` / `.txt`) — admin only
POST	`/upload-user-policy`	multipart `userDID` + `file`

11.7 Requests & user administration

METHOD	ENDPOINT	PURPOSE
GET	`/agents-creation-requests-list?page=`	ADMIN every creation request in the org
GET	`/agents-creation-requests-list-user?page=`	The caller's own creation requests
POST	`/agents-creation-requests-create`	multipart `agentName` (required), `agentID?`, `requestInfo?`, `policy` (file or plain text) → `{ requestID }`
POST	`/agents-creation-requests-edit`	`{ requestID, agentName, policy, requestInfo?, agentID? }` → `{ requestID }`
POST	`/agent-creation-request-result-submit`	ADMIN `{ requestID, status: "approved"|"rejected" }`
GET	`/agent-access-requests-list-org?page=`	ADMIN org-wide access requests
GET	`/agent-access-requests-list-user?page=`	The caller's own access requests
POST	`/agent-access-request-create`	`{ agentDID, agentName?, requestInfo? }` → `{ requestID }`
POST	`/agent-access-request-submit`	ADMIN `{ requestID, status }`
POST	`/create-user`	ADMIN `{ did, name?, email, password, orgID }`. `nft_id` and `api_key` are generated server-side; a blank name auto-assigns `user_1`, `user_2`, …

Request record shape (both queues):

```
{ requestID, requestType: "deploy_agent" | "agent_access",
  policy, creatorDID, agentDID, agentName, requestInfo,
  status: "pending" | "approved" | "rejected", createdAt }
```

Proposed, not guaranteed. `/user-access-list`, `/admin-grant-agent-access` and `/admin-revoke-agent-access` are wired in the client but may not exist on every backend build. Mark them clearly as provisional in the docs.

12. Authentication & authorization

12.1 Two sign-in paths

	USER	ADMIN
Endpoint	`POST /login` on the dashboard API	`POST /login` on the agent-admin API
Credentials	email + password	username + password
Response	Full object: token, did, email, org, api_key, is_admin, access list	A bare JWT string in `data`
Session build	Fields copied straight from the response	JWT decoded; `sub` without `email` implies admin

12.2 Token handling

- Stored at `localStorage["agentdna.token"]`; the decoded user at `localStorage["agentdna.user"]`.
- `VITE_DEV_TOKEN` acts as a fallback when storage is empty — a development convenience that must not be set in production builds.
- Expiry is enforced client-side from the `exp` claim, re-checked every 30 seconds.
- A 401 raises `ApiError(401)`. By default `skipLogoutOn401` is `true`, so a single failing panel does not eject the user; endpoints that opt out clear the session and fire the unauthorized handler.

12.3 Registration flows

User self-registration:

```
POST /send-otp { email }                                    → OTP emailed, 300s resend countdown
POST /register-user { name?, email, password, orgID, otp } → { api_key, … }
POST /login { email, password }                             → session established
GET  /user-profile                                          → patch name + org onto the session
```

Admin registration spans both services:

```
POST /create-admin    { username, email, orgID, password, otp }  → { did }   (dashboard API)
POST /register-admin  { did, org_id }                            → whitelist the DID
POST /login           { username, password }                     → JWT       (admin API)
GET  /admin-profile                                              → patch name + org
```

12.4 Authorization surface

- `ProtectedRoute` redirects unauthenticated visitors to `/`, preserving the attempted path, and supports an `adminOnly` flag that bounces non-admins to `/dashboard`.
- Admin-only UI: approve/reject on both request queues, the Users tab, Add User, per-user access management, and agent-policy replacement.
- The pending-requests badge only loads for admins.

> Client-side gating is a UX affordance, not a security control. Every restricted operation must be enforced server-side; the docs should say so explicitly.

13. Key end-to-end flows

13.1 Sign in

```
Landing (/) → pick role → submit credentials
  → POST /login (dashboard or admin API)
  → store token + user in localStorage
  → background GET /user-profile | /admin-profile → patch name + org
  → DirectoryProvider walks agents/tools/users → DID→name map
  → IntentNumbersProvider walks intents → stable intent numbers
  → redirect to the originally attempted path, else /dashboard
```

13.2 Request an agent deployment

```
User: /requests → "New request" → AgentRequestModal
  fields: agentName (required), agentID?, requestInfo?, policy file (.md/.txt) or text
  → POST /agents-creation-requests-create  (multipart) → { requestID }
  → row appears as "pending"; the user may edit it while pending
     via POST /agents-creation-requests-edit

Admin: sidebar badge shows the pending count
  → /requests → Creation tab → review name, requester, policy
  → Approve → POST /agent-creation-request-result-submit { requestID, "approved" }
  → DeployAgentModal shows loading → success | error
  → list reloads; the new agent appears under /agents
```

13.3 Request access to an existing agent

```
User: /agents → pick an agent → "Request access" → AccessRequestModal
  → POST /agent-access-request-create { agentDID, agentName?, requestInfo? }
Admin: /requests → Org access requests
  → POST /agent-access-request-submit { requestID, "approved" | "rejected" }
  → the agent joins the user's agent_access_list
```

13.4 Investigate an intent

```
/dashboard or /intents → click an intent
  → /intents/:id
      GET /intent-info                        header + status + provenance
      GET /interactions-list?intentID=&page=   ledger (all pages walked to derive participants)
      → Interactions tab: row → interaction drawer (full IDs, threat, block type)
      → Agents & Apps tab: derived participants → agent page or tool drawer
  → "View flow" → /graph/:id
      GET /intent-diagram        preferred: real messages + typed hops
      GET /intent-block-data     fallback: signed block chain → trace spans
      → animated canvas + step rail + TraceInspector (input/output/model/signature/raw JSON)
  → "Export PDF" → jsPDF report generated in the browser
```

13.5 Update an agent policy

```
Admin: /agents/:id → "Edit policy" → EditAgentPolicyModal → pick .md/.txt
  → POST /upload-agent-policy (multipart agentDID + file)
  → Policy History tab reloads: GET /agent-policy-history
  → click a version: GET /agent-policy-update?agentDID=&updateID= → ViewPolicyModal
```

13.6 Export reports

EXPORT	WHERE	CONTENTS
Agent PDF	Agent detail	Summary, score, interactions, intents, policy history
Intent PDF	Intent detail	Intent header, interaction list, participant rollup
List PDF	Agents & Apps	The visible agent or app table, with status colouring
Dashboard CSV	Home	Summary block plus the full agent list

All exports are generated client-side (jsPDF for PDF, a Blob download for CSV) at A4, 40 pt margins, with a navy/blue/red palette matching the UI. No data leaves the browser.

14. Configuration & environment

VARIABLE	REQUIRED	DESCRIPTION
`VITE_API_BASE_URL`	Yes	Base URL of the dashboard middleware API, e.g. `http://host:9000/dashboard/v1/`. A trailing slash is stripped. Falls back to `http://localhost:9000`.
`VITE_ADMIN_API_BASE_URL`	Yes	Base URL of the agent-admin API, e.g. `http://host:8001/agent-admin/v1`. Falls back to `http://localhost:8000/agent-admin/v1`.
`VITE_DUMMY`	No	`true` / `1` runs entirely on mock data with no backend.
`VITE_PORT`	No	Dev-server port, default `4009` (`strictPort` is on, so a busy port fails loudly).
`VITE_DEV_TOKEN`	No	Fallback bearer token when `localStorage` has none. Development only.

Resolution order

```
window.__ENV__.VITE_API_BASE_URL     ← runtime, written by the Docker entrypoint
  ?? import.meta.env.VITE_API_BASE_URL ← build time, baked in by Vite
  ?? "http://localhost:9000"           ← last-resort default
```

This is what makes one image deployable to many environments: `index.html` loads `/env-config.js` before the bundle, so the runtime value is present before the first request. Note that the admin base URL currently reads only from `import.meta.env`, so changing it requires a rebuild — worth flagging in the docs.

Dev-server `allowedHosts` is pinned to `dashboard-dev.agentdna.io` and `dashboard.agentdna.io`.

15. Build, deployment & demo mode

15.1 Local development

```
npm install
cp .env.sample .env      # fill in the two base URLs
npm run dev              # http://localhost:4009
npm run build            # tsc -b && vite build → dist/
npm run preview
npm run lint
```

15.2 Docker

```
# Build (multi-stage: node:22-alpine → nginx:1.27-alpine)
docker build -t agentdna-dashboard -f docker/Dockerfile .

# Run — configuration happens at runtime, not build time
docker run -p 80:80 \
  -e VITE_API_BASE_URL=http://your-backend-ip:9000/dashboard/v1/ \
  -e VITE_ADMIN_API_BASE_URL=http://your-backend-ip:8001/agent-admin/v1 \
  -e VITE_DUMMY=false \
  agentdna-dashboard
```

`docker/entrypoint.sh` writes the runtime config before starting nginx:

```
window.__ENV__ = {
  VITE_API_BASE_URL: "…",
  VITE_ADMIN_API_BASE_URL: "…",
  VITE_DUMMY: "false"
};
```

The nginx layer serves `dist/` with an SPA fallback (`try_files $uri $uri/ /index.html`), gzip for text assets, and `Cache-Control: public, immutable` with a one-year expiry for hashed assets.

> **Host networking.** On macOS and Windows, a backend on the same machine is reachable as `host.docker.internal`, not `localhost`. On Linux, use the host's LAN IP.

15.3 Dummy / demo mode

With `VITE_DUMMY=true` the transport layer never reaches the network. Every request is offered to `src/data/dummyRouter.ts`, which serves 10-item pages from `src/data/dummy.json` and logs each intercepted call as `[DUMMY <METHOD> <path>]`. Auth is stubbed with a fixed demo user, so sign-in succeeds with any credentials. The volume chart is bucketed from the seeded interaction timestamps — anchored on the latest seeded time rather than "now", so the window is always full — and layered with a deterministic baseline so the chart is never flat.

Useful for demos, design review, and front-end work while the backend is unavailable.

16. Known gaps, stubs & docs structure

16.1 Stubs and client-side compensations

AREA	CURRENT STATE
Heatmap (`fetchHeatmap`)	Returns an empty array — no endpoint yet
Logs (`fetchLogs`)	Returns an empty array — no endpoint yet
Alerts	No dedicated endpoint; threats are the interaction feed filtered client-side, so counts reflect the fetched page, not the org
Intent participants	Derived client-side by walking every page of the intent's interactions — expensive for large intents
Directory & intent numbering	Walk every page of four collections on login; fine for current org sizes, a scaling concern later
Intent list filters	Applied to the current page only
Agent/tool classification	Inferred from the `bafy` DID prefix where the backend does not label the side
Status thresholds	Hard-coded in the client (agents > 5 threats, tools > 4)
Access management endpoints	Three endpoints wired but marked proposed
Unrouted pages	`AlertsPage` and `LandingPage` exist but are not in the route table
Admin base URL	Build-time only — not runtime-configurable like the main API URL

16.2 Proposed Docusaurus sidebar

```
Introduction
  ├─ What is the AgentDNA Dashboard          (§1)
  ├─ Concepts & glossary                     (§2)
  └─ Quick start                             (§15.1)

Architecture
  ├─ System overview                         (§4)
  ├─ Application structure                   (§5, §6)
  ├─ Routing                                 (§7)
  ├─ State management                        (§8)
  └─ Data layer & types                      (§10)

User guide
  ├─ Signing in & registration               (§9.1, §12.3)
  ├─ Home dashboard                          (§9.2)
  ├─ Intents & intent detail                 (§9.3, §9.4)
  ├─ Agents & Apps                           (§9.5, §9.6, §9.7)
  ├─ Users                                   (§9.8)
  ├─ Interactions ledger                     (§9.9)
  ├─ Flow visualization                      (§9.10)
  ├─ Requests & approvals                    (§9.11)
  ├─ Profile & API keys                      (§9.12)
  └─ Exporting reports                       (§13.6)

API reference
  ├─ Conventions                             (§11.1)
  ├─ Authentication & account                (§11.2)
  ├─ Metrics                                 (§11.3)
  ├─ Lists                                   (§11.4)
  ├─ Detail                                  (§11.5)
  ├─ Policy                                  (§11.6)
  └─ Requests & administration               (§11.7)

Guides
  ├─ Deploy an agent (end to end)            (§13.2)
  ├─ Grant agent access                      (§13.3)
  ├─ Investigate an intent                   (§13.4)
  └─ Manage agent policies                   (§13.5)

Operations
  ├─ Configuration                           (§14)
  ├─ Docker deployment                       (§15.2)
  ├─ Demo mode                               (§15.3)
  └─ Known limitations                       (§16.1)
```

16.3 Writing guidance for the docs

- **Use the UI's vocabulary.** The interface says *Apps*; the code says `Tool`. User-facing docs should say Apps and note the equivalence once.
- **Be honest about derived data.** Where the client computes something the backend does not provide (participants, threat filtering, intent numbering), say so — it prevents bug reports about numbers that "don't match".
- **Split by audience.** Admin-only capabilities are a meaningful subset; either badge them inline or give admins their own section.
- **Keep the API reference generated-looking.** Consistent method / path / auth / request / response tables per endpoint, as in §11.

- **Screenshots.** Demo mode (`VITE_DUMMY=true`) produces a fully populated dashboard with no backend — the cheapest way to capture consistent screenshots for every page.